Taller 2: Manipulación de Datos en R I

Métodos Cuantitativos II

M. Constanza Ayala

maria.ayala@uv.cl

Agenda una reunión aquí

Escuela de Sociología - Universidad de Valparaíso

25 de marzo, 2026

Pasemos lista mientras descargan los materiales y dejamos todo listo ✨

📂 **Paso 1:** Abrimos nuestro proyecto con doble clic en el archivo `.Rproj`

- Si no tienen proyecto, creamos uno: - `File > New Project > New Directory > New Project` - Creamos carpetas: `data`, `scripts`, `figures`, `output`

💾 **Paso 2:** Movemos el archivo **"Script Taller 2 Métodos Cuantitativos II.R"** a la carpeta `scripts`

📎 **Paso 3:** Movemos las bases **base_93.Rds** y **Base Evaluaciones ELPI III_reducida.RData** a la carpeta `data`
💡 ¡Recuerda mantener el nombre original del archivo para facilitar su lectura en R!

> **Todos los materiales los pueden descargar en este link:** Materiales clases

Evaluación 1

Hoy tenemos nuestra **primera evaluación individual** en R. Debes entregar lo siguiente:

- Proyecto de R (.Rproj).
- El script (.R) que contiene todos los ejercicios individuales solicitados en este taller. Debes crear un script nuevo, no utilices el script de actividades entregado en la clase.
  - En ese script, incluye un título, tu nombre, correo y fecha. Por ejemplo:

```
# Script Taller 2 Métodos Cuantitativos II ------------------------------------------
# M. Constanza Ayala (maria.ayala@uv.cl)
# 25-03-2026
```

Plazo de entrega: hoy hasta las 23:59 hrs. en el Aula Virtual.

> En caso de utilizar IA, debe reportarlo en un archivo Word, indicando la herramienta y los prompts usados. El uso no reportado será calificado con nota mínima.

Objetivos de la clase

- Introducir principios del **uso de bases de datos** en R

- Introducir el paquete **`dplyr` para manipulación de datos**

- **Aplicar funciones** clave: `select()`, `filter()`, `arrange()`

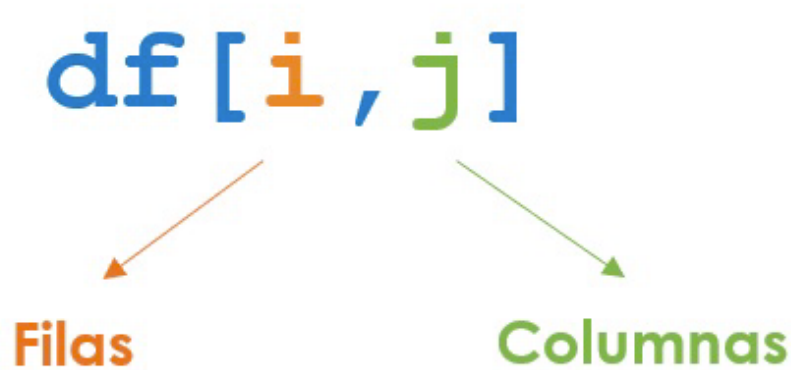
¿Qué vimos la semana pasada en R?

✅ Creamos objetos y vectores simples
✅ Exploramos la interfaz y creamos proyectos
✅ Aprendimos sobre **scripts** y **librerías**

💡 ¡Hoy damos paso a la **manipulación de datos**!

Trabajo con Bases de Datos

Recordemos que las **bases de datos** son objetos bidimensionales. Por lo que operan bajo la lógica:




```r
head(mtcars)
```

```
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

df[, 2]

```
mtcars[,2]
```

```
[1] 6 6 4 6 8 6 8 4 4 6 6 8 8 8 8 8 4 4 4 4 8 8 8 8
4 4 4 8 6 8 4
```

df[2,]

```
mtcars[2,]
```

```
               mpg cyl disp  hp drat    wt  qsec
Mazda RX4 Wag   21   6  160 110  3.9 2.875 17.02
```

df[2, 2]

```
mtcars[2,2]
```

```
[1] 6
```

Por otra parte, los objetos se pueden conservar encadenados a través de otro objeto con el operador **$**:

```
mtcars$cyl
```

```
[1] 6 6 4 6 8 6 8 4 4 6 6 8 8 8 8 8 8 4 4 4 4 8 8 8 8 4 4 4 8 6 8 4
```

- Esto nos permite “llamar” una determinada variable del objeto base de datos.

Extensión de las bases de datos

En R podemos trabajar con archivos de distintas **extensiones**, por ejemplo:

- `.RData` o `.rds` → propios de R
- `.csv` → valores separados por coma o punto-coma
- `.xlsx` → hojas de cálculo de Excel
- `.sav` → archivos de SPSS
- `.dta` → archivos de Stata

Cada formato requiere funciones o paquetes específicos para ser importado.

Importar datos en R

En R base, existen varias funciones para **cargar datos** desde distintos formatos:

- `load("archivo.RData")`
- `readRDS("archivo.rds")`
- `read.csv("archivo.csv")`
- `read.table("archivo.txt")`

```
#Veamos un ejemplo con base en formato R

load("data/Base Evaluaciones ELPI III_reducida.RData")
```

Paquetes para importar datos

Además de las funciones básicas, existen **paquetes especializados** que permiten trabajar con múltiples formatos, por ejemplo, los más usados son:

- **haven** → importar archivos de SPSS (`.sav`), Stata (`.dta`) y SAS (`.sas7bdat`)

```r
library(haven)
data_spss <- read_sav("archivo.sav")
data_stata <- read_dta("archivo.dta")
```

- **rio** → paquete que reconoce automáticamente el tipo de archivo

```r
library(rio)
data_xlsx <- import("archivo.xlsx")
data_spss <- import("archivo.sav")
data_stata <- import("archivo.dta")
```

Exportar bases de datos

Con la siguiente función base de R podemos exportar una base de datos en formato RData:

```
save(data, file="output/data_arrange3.RData")
```

Para exportar a otros formatos, en relación con sus funciones de importación:

```
readRDS("archivo.rds") → saveRDS(data, file = "archivo.rds")

read.csv("archivo.csv") → write.csv(data, file = "archivo.csv")

read.table("archivo.txt") → write.table(data, file = "archivo.txt")
```

Podemos hacer lo mismo con paquetes **haven** y **rio**

- **haven**

```r
library(haven)

write_sav(data_spss,  "archivo.sav")
write_dta(data_stata, "archivo.dta")
```

- **rio**

```r
library(rio)

export(data_xlsx, "archivo.xlsx")
export(data_spss, "archivo.sav")
export(data_stata, "archivo.dta")
```

Explorar la base de datos

```r
# Podemos ver los componentes del objeto
dim(data)   # Observaciones y variables
```

```
[1] 15827    10
```

```r
names(data) # Nombre de nuestras variables
```

```
 [1] "folio"          "espanel"        "sexo"           "nivel_edu_niño"
 [5] "edad_mesesr"    "idregion"       "wm_pb_pa"       "wm_pb_fd"
 [9] "wm_pb_cc"       "pscs_pb_sp"
```

```r
str(data)   # Visor de nuestras variables
```

```
'data.frame':   15827 obs. of  10 variables:
 $ folio         : num  1e+05 1e+05 1e+05 1e+05 1e+05 ...
  ..- attr(*, "label")= chr "folio: id entrevista"
  ..- attr(*, "format.stata")= chr "%10.0g"
 $ espanel       : num  1 1 1 1 1 1 1 1 1 1 ...
  ..- attr(*, "label")= chr "Tipo de muestra"
  ..- attr(*, "format.stata")= chr "%10.0g"
  ..- attr(*, "labels")= Named num [1:2] 0 1
  .. ..- attr(*, "names")= chr [1:2] "Refresco" "Panel"
 $ sexo          : num  2 1 1 1 1 2 1 1 1 2 ...
  ..- attr(*, "label")= chr "Sexo del niño seleccionado"
  ..- attr(*, "format.stata")= chr "%10.0g"
```

```
..- attr(*, "labels")= Named num [1:2] 1 2
      attr(*, "names")= chr [1:2] "Masculine" "Femenine"
```

```r
head(data) # Primeras 6 observaciones
```

```
   folio espanel sexo nivel_edu_niño edad_mesesr idregion wm_pb_pa wm_pb_fd
1 100011       1    2              4         144       15       33       48
2 100081       1    1              4         144       15       38       78
3 100091       1    1              4         146       15       34       43
4 100121       1    1              4         144       15       33       33
5 100131       1    1              4         143       15       41       80
6 100151       1    2              4         147       15       37       69
  wm_pb_cc pscs_pb_sp
1       18         41
2       21         37
3       15         30
4       13         38
5       25         39
6       18         37
```

```r
tail(data) # Últimas 6 observaciones
```

```
       folio espanel sexo nivel_edu_niño edad_mesesr idregion wm_pb_pa
wm_pb_fd
15822 745001       0    1              1          31        5       NA
NA
15823 745061       0    2              1          59        6       NA
NA
15824 746041       0    1              0          29        2       NA
NA
```

15825	746151	0	1	0	20	2	NA
NA							
15826	746221	0	2	4	78	5	NA
NA							
15827	746231	0	1	1	49	5	NA
NA							

```r
# Lógica fila, columna
data[1] # primera columna del data frame
```

```
       folio
1     100011
2     100081
3     100091
4     100121
5     100131
6     100151
7     100161
8     100171
9     100211
10    100221
11    100271
12    100291
13    100361
```

```r
data[1,] #primera fila
```

```
   folio espanel sexo nivel_edu_niño edad_mesesr idregion wm_pb_pa wm_pb_fd
1 100011       1    2               4         144       15       33       48
  wm_pb_cc pscs_pb_sp
1       18         41
```

```r
data[,1] #primera columna
```

```
  [1] 100011 100081 100091 100121 100131 100151 100161 100171 100211 100221
 [11] 100271 100291 100361 100391 100401 100411 100461 100471 100511 100531
 [21] 100541 100551 100561 100611 100621 100651 100661 100691 100701 100731
 [31] 100761 100801 100821 100861 100881 100931 100941 101041 101061 101081
 [41] 101101 101141 101171 101181 101201 101231 101241 101301 101321 101341
 [51] 101351 101421 101451 101461 101481 101521 101541 101571 101581 101641
 [61] 101651 101721 101751 101761 101781 101791 101801 101811 101821 101861
 [71] 101891 101901 101911 101931 101941 101961 101991 102031 102061 102111
 [81] 102121 102131 102141 102161 102171 102181 102191 102231 102261 102291
 [91] 102301 102371 102391 102441 102451 102471 102511 102611 102701 102731
[101] 102741 102781 102791 102801 102811 102861 102901 102931 102941 102971
[111] 102991 103001 103031 103051 103061 103091 103111 103121 103161 103181
[121] 103201 103261 103281 103301 103321 103341 103361 103391 103401 103431
[131] 103441 103451 103461 103471 103501 103541 103561 103601 103641 103661
```

```r
data[1,1] #elemento ubicado en la primera fila y primera columna
```

```
[1] 100011
```

```r
data[1:6,1:6] #filas de la 1 a la 6 y las columnas de la 1 a la 6
```

```
   folio espanel sexo nivel_edu_niño edad_mesesr idregion
1 100011       1    2              4         144       15
2 100081       1    1              4         144       15
3 100091       1    1              4         146       15
4 100121       1    1              4         144       15
5 100131       1    1              4         143       15
6 100151       1    2              4         147       15
```

```r
data[,c("sexo","nivel_edu_niño")] #todas las filas de las columnas
```

```
      sexo nivel_edu_niño
1        2              4
2        1              4
3        1              4
4        1              4
5        1              4
6        2              4
7        1              4
8        1              4
9        1              4
10       2              4
11       1              4
```

12	2	4
13	1	4

Actividad 1 para ejercicio

1. Cargar los paquetes `dplyr` y `haven` utilizando la función `library()` en una sección del script llamada `Paquetes`.
2. Importar la base de datos `base_93.Rds` utilizando la función `readRDS()` en una sección del script llamada `Base de datos`.
3. Explorar la base de datos en una sección llamada `Exploración base de datos` utilizando las funciones `dim()`, `names()`, `str()`, `head()` y `tail()`. Luego, responde en comentarios de R (`#`) las siguientes preguntas:
   - ¿Cuántas observaciones y variables tiene la base?
   - ¿Hay variables que parezcan ser identificadores únicos de cada caso? ¿Cuáles?
4. Incorporar en la exploración la lógica de filas y columnas, identificando:
   - Valores de una variable (columna)
   - Valores de un caso (fila)
   - Un valor específico (fila y columna)
   - Por ejemplo: `data[1,1]`

De cargar a transformar datos

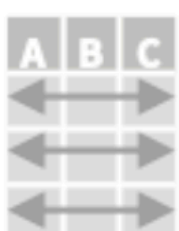
- Ya vimos cómo **importar** bases de datos en R.
- El siguiente paso es **trabajar con ellas**: explorarlas, transformarlas y limpiarlas.
- Para esto, usaremos un conjunto de funciones del paquete `dplyr`.

¿Qué es dplyr?

- Parte de **tidyverse** (colección de paquetes de R)
- Pensado para **transformar y limpiar datos**
- Permite escribir **código legible y encadenado** con %>%

dplyr functions work with pipes and expect **tidy data**. In tidy data:

&



Each **variable** is in its own **column**

Each **observation**, or **case**, is in its own **row**

pipes

x %>% f(y) becomes **f(x, y)**

Ocupemos %>%

```r
# Cargamos la librería
library(dplyr)

# Exploremos los datos con dplyr
data %>% glimpse() # Vista previa
```

```
Rows: 15,827
Columns: 10
$ folio          <dbl> 100011, 100081, 100091, 100121, 100131, 100151, 100161,…
$ espanel        <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1…
$ sexo           <dbl> 2, 1, 1, 1, 1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 2, 2, 1, 1, 2…
$ nivel_edu_niño <dbl> 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4…
$ edad_mesesr    <dbl> 144, 144, 146, 144, 143, 147, 146, 143, 143, 147, 142, …
$ idregion       <dbl> 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15,…
$ wm_pb_pa       <dbl> 33, 38, 34, 33, 41, 37, 33, 34, 32, 38, 37, 40, 23, 40,…
$ wm_pb_fd       <dbl> 48, 78, 43, 33, 80, 69, 49, 37, 60, 63, 61, 43, 31, 54,…
$ wm_pb_cc       <dbl> 18, 21, 15, 13, 25, 18, 14, 11, 21, 25, 15, 20, 4, 19, …
$ pscs_pb_sp     <dbl> 41, 37, 30, 38, 39, 37, 41, 45, 37, 28, 41, 44, 38, 44,…
```

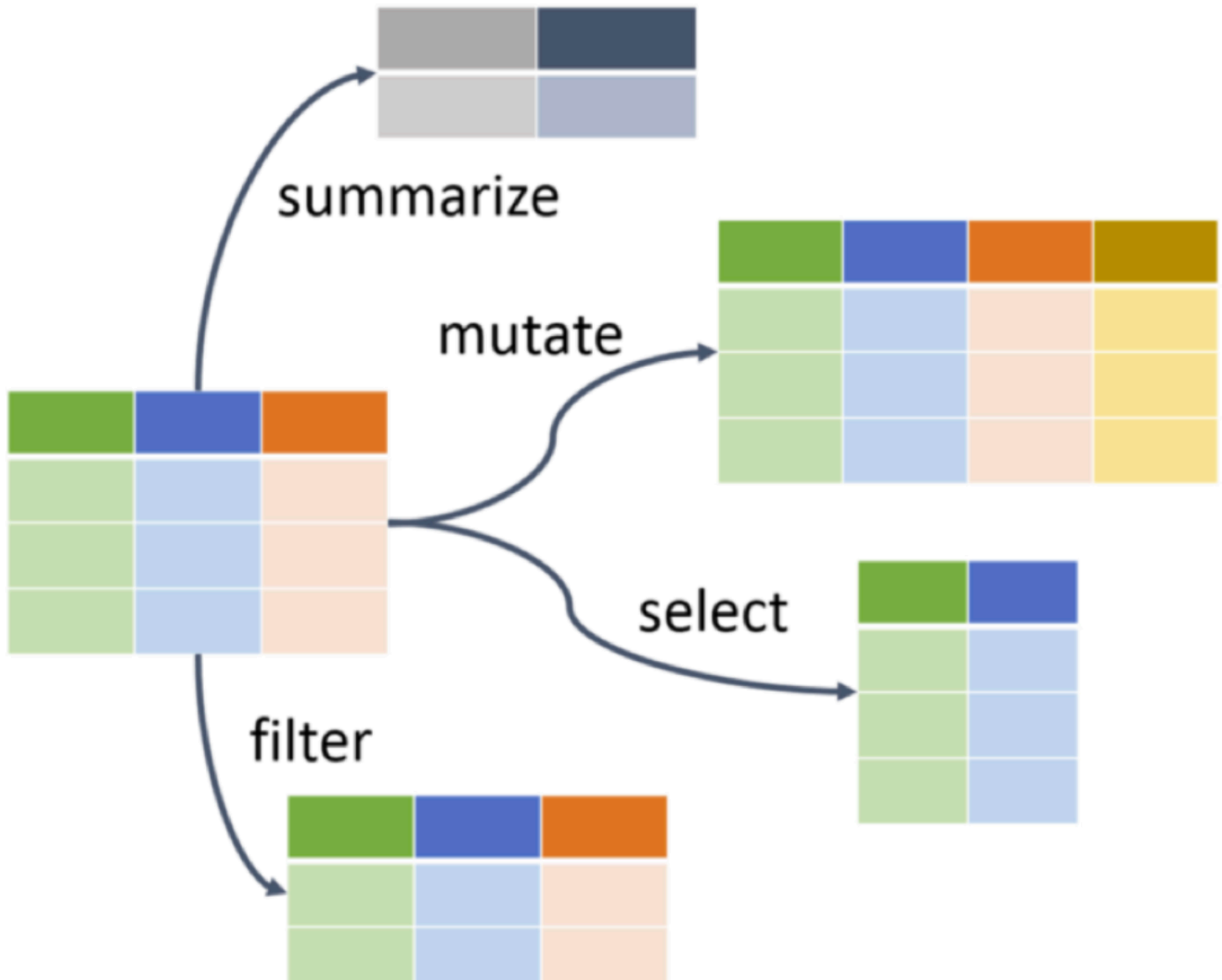
```r
data %>% head()    # Primera 6 observaciones
```

```
   folio espanel sexo nivel_edu_niño edad_mesesr idregion wm_pb_pa wm_pb_fd
1 100011       1    2              4         144       15       33       48
2 100081       1    1              4         144       15       38       78
3 100091       1    1              4         146       15       34       43
4 100121       1    1              4         144       15       33       33
5 100131       1    1              4         143       15       41       80
6 100151       1    2              4         147       15       37       69
  wm_pb_cc pscs_pb_sp
1       18         41
2       21         37
3       15         30
4       13         38
```

```r
data %>% colnames() # Nombres columnas/variables
```

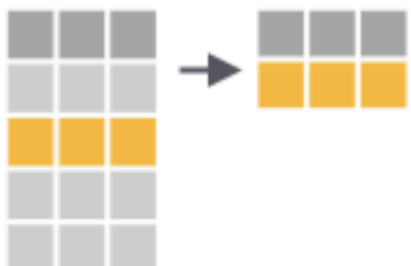
```
[1] "folio"          "espanel"        "sexo"           "nivel_edu_niño"
[5] "edad_mesesr"    "idregion"       "wm_pb_pa"       "wm_pb_fd"
[9] "wm_pb_cc"       "pscs_pb_sp"
```

Manipulación de datos con dplyr



Transformación usando vectores

Row functions return a subset of rows as a new table.



filter(.data, …, .preserve = FALSE**)** Extract rows that meet logical criteria.
filter(mtcars, mpg > 20)

select(.data, …**)** Extract columns as a table.
select(mtcars, mpg, wt)

arrange(.data, …, .by_group = FALSE**)** Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
arrange(mtcars, mpg)
arrange(mtcars, desc(mpg))

filter()

- Devuelve filas que cumplen con al menos una condición lógica
- Puedes usar operadores lógicos: ==, >, <, ! =, &, |

```r
#Hombre
data_hombre <- data %>%
  filter(sexo=="Hombre")
dim(data_hombre)
```

```
[1] 8095   10
```

```r
#Mujer
data_mujer <- data %>%
  filter(sexo=="Mujer")
dim(data_mujer)
```

```
[1] 7732   10
```

- Puedes aplicar más de una condición:

```r
data_mat1 <- data %>%
  filter(wm_pb_pa >=36 & wm_pb_fd==36)
dim(data_mat1)
```

```
[1] 55 10
```

```r
data_mat2 <- data %>%
  filter(wm_pb_pa >=36 | wm_pb_fd==36)
dim(data_mat2)
```

```
[1] 3250   10
```

Operadores lógicos en R

- Sirven para hacer comparaciones y construir condiciones en funciones

Operador	Significado	Ejemplo
`==`	igual a	`species == "Human"`
`!=`	distinto de	`gender != "male"`
`>`	mayor que	`mass > 80`
`<`	menor que	`height < 180`
`&`	**y** lógico (ambas)	`species == "Human" & mass > 75`
`|`	**o** lógico (una u otra)	`species == "Human" | species == "Droid"`

select()

- Devuelve solo la o las columnas que especifiques

```r
data1 <- data %>%
  select(1:3)
data1 %>% colnames()
```

```
[1] "folio"   "espanel" "sexo"
```

```r
data2 <- data %>%
  select("folio", "wm_pb_pa")
data2 %>% colnames()
```

```
[1] "folio"    "wm_pb_pa"
```

```r
data3 <- data %>%
  select("folio", "wm_pb_pa":"wm_pb_cc")
data3 %>% colnames()
```

```
[1] "folio"    "wm_pb_pa" "wm_pb_fd" "wm_pb_cc"
```

```r
data4 <- data %>%
  select(1,edad_mesesr,sexo,idregion,wm_pb_pa:wm_pb_cc)
data4 %>% colnames()
```

```
[1] "folio"       "edad_mesesr" "sexo"        "idregion"    "wm_pb_pa"
[6] "wm_pb_fd"    "wm_pb_cc"
```

- También puedes usar: select(-columna) para excluir

```r
data5 <- data %>%
  select(-sexo)
data5 %>% colnames()
```

```
[1] "folio"          "espanel"        "nivel_edu_niño" "edad_mesesr"
[5] "idregion"       "wm_pb_pa"       "wm_pb_fd"       "wm_pb_cc"
[9] "pscs_pb_sp"
```

arrange()

- Ordena filas por una o más variables

```r
data_arrange1 <- data4 %>%
  arrange(wm_pb_pa)  # Por defecto el orden es creciente
head(data_arrange1, n=15)
```

```
    folio edad_mesesr   sexo idregion wm_pb_pa wm_pb_fd wm_pb_cc
1  102741         114 Hombre       15        0        0        0
2  124551         143 Hombre        3        0        0        0
3  127371         107 Hombre        3        0       19        0
4  135961         113  Mujer        4        0        0        0
5  141981         111 Hombre        5        0        0        0
6  166991         142  Mujer        5        0        0       NA
7  177871         117  Mujer        6        0        0        8
8  190951         129 Hombre        7        0        4       16
9  193671         145  Mujer        7        0        0        1
10 193911         142 Hombre        7        0        0        0
11 247021         121 Hombre        9        0        0        0
12 276351         140 Hombre       12        0        0        0
13 278341         149 Hombre       13        0       15        1
```

- Usa desc() para ordenar de forma descendente

```r
data_arrange2 <- data4 %>%
  arrange(desc(wm_pb_pa)) # Decreciente
head(data_arrange2, n=15)
```

```
    folio edad_mesesr   sexo idregion wm_pb_pa wm_pb_fd wm_pb_cc
1  166841         146  Mujer        5       62       90       30
2  168121         111 Hombre        5       62       60       15
3  167351         132  Mujer        5       59       90       39
4  340901         110  Mujer       13       59       70       32
5  167261         136 Hombre        5       58       80       31
6  353591         133 Hombre       13       58       85       23
7  359991         136  Mujer       13       58       50       19
8  112361         145 Hombre        4       57       69       16
9  166821         147 Hombre        5       57      120       28
10 167011         141 Hombre        5       57       70       29
11 116031         146 Hombre        2       56       37       14
12 166901         145 Hombre        5       56       67       35
13 166961         144 Hombre        5       56       53       31
```

- También puedes ordenar por varias columnas

```r
data_arrange3 <- data4 %>%
  arrange(wm_pb_pa,wm_pb_fd)  # Utilizando dos variables
head(data_arrange3, n=15)
```

```
    folio edad_mesesr   sexo idregion wm_pb_pa wm_pb_fd wm_pb_cc
1  102741         114 Hombre       15        0        0        0
2  124551         143 Hombre        3        0        0        0
3  135961         113  Mujer        4        0        0        0
4  141981         111 Hombre        5        0        0        0
5  166991         142  Mujer        5        0        0       NA
6  177871         117  Mujer        6        0        0        8
7  193671         145  Mujer        7        0        0        1
8  193911         142 Hombre        7        0        0        0
9  247021         121 Hombre        9        0        0        0
10 276351         140 Hombre       12        0        0        0
11 278571         146  Mujer       13        0        0       10
12 366311         131 Hombre       13        0        0        0
13 371491         131 Hombre       13        0        0        0
```

Actividad 2 para ejercicio

1. Crea una base de datos llamada `data_reducida` aplicando los siguientes filtros con `filter()` en una sección del script llamada `Filtros`:
   - Mujeres (2023) (`sexo`, categoría `2`) y
   - Personas entre 20 y 60 años (`edad`)
   - Explora la nueva base de datos con `dim()`
2. Selecciona las siguientes variables con `select()` en una sección del script llamada `Selección de variables`: `id_bu`, `id_bu_encuesta`, `region_3`, `zona_u_r`, `gse`, `edad`, `estado_civil_g`. Luego, explora la base de datos con `glimpse()`.
3. Ordena la base de datos de forma ascendente según las variables `region_3` e `id_bu_encuesta` usando `arrange()` en una sección del script llamada `Arrange`. El objetivo es que, dentro de cada región, los casos queden ordenados según su id.
4. Guarda la base de datos editada en formato RData en la carpeta `output` con el nombre `data_cep_93`.

❓ Preguntas y aclaraciones

💬 **¿Dudas sobre lo visto hoy?**

- Tómense un momento para reflexionar y compartir preguntas sobre el contenido.
- Espacio para responder inquietudes y aclarar conceptos clave.

Resumen clase de hoy

✅ Cargamos una base de datos y la exploramos

✅ Introdujimos `dplyr` para manipular datos

✅ Usamos funciones clave:

- `filter()` para seleccionar filas según condiciones
- `select()` para elegir o eliminar columnas
- `arrange()` para ordenar los datos

📅 Próxima sesión (miércoles)

- Continuaremos con manipulación de datos con `dplyr`

📚 Sugerencias lecturas para reforzar R

- Wickham & Grolemund. R for Data Science (acceso en línea biblioteca, en español: https://es.r4ds.hadley.nz/)
- AnalizaR Datos Políticos
- YaRrr! The Pirate's Guide to R